

Programación

EXAMEN PRÁCTICO TEMA 6 – POO avanzada

(27/02/2026)



LEE ATENTAMENTE LAS SIGUIENTES INSTRUCCIONES ANTES DE EMPEZAR:



- **Recopila en un documento de texto las evidencias de todo el examen.** Guárdalo de vez en cuando para no perder el avance de tu trabajo.
- Cuando termines, **pásalo a PDF y sube el documento creado a la entrega de AULES.**

PARTE 1: Configuración del entorno

1. Crea un nuevo repositorio llamado “EXAMEN_UD6_drift” desde *SourceTree*. El repositorio debe crearse en local y tener su espejo en remoto, por lo tanto, sincronízalo con *GitHub*.

Pega a continuación la URL a tu nuevo repositorio de *GitHub*:

2. Crea un nuevo proyecto *Java* (*Maven*) con *IntelliJ* -o el IDE que utilices- dentro del repositorio que acabas de crear. Llámalo “EXAMEN UD6 DRIFT”.
3. Crea en el proyecto un paquete nuevo llamado “drift_spain_series”.

Sincroniza los cambios en tu repositorio remoto.

PARTE 2: Resolución de problemas



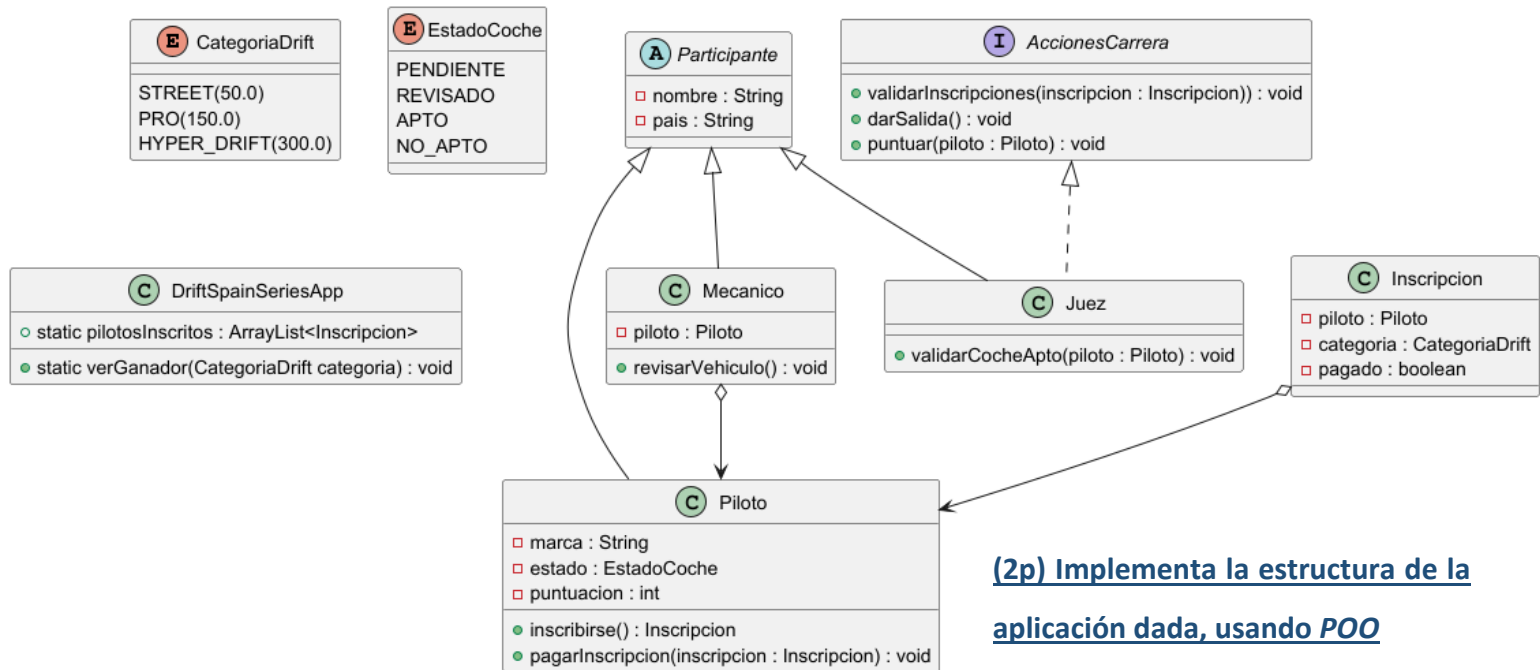
Programa en *Java* la solución a los siguientes ejercicios. Usa el proyecto que te acabas de crear en el apartado anterior.

Si no has conseguido crearlo correctamente, utiliza alguno de los proyectos que ya tenías para los ejercicios de clase y pega la URL de *GitHub* del repositorio al que vas a subir los cambios.

App para gestionar el evento *Drift Spain Series* en Valencia

El rugido de los motores y el olor a neumático quemado están a punto de invadir Valencia. Los próximos 25 y 26 de abril, la ciudad se convertirá en el epicentro del automovilismo extremo con la llegada de las *Drift Spain Series*. Ante la magnitud del evento y el volumen de participantes esperado, la organización ha decidido dejar atrás los registros en papel y profesionalizar la gestión de la prueba mediante una aplicación avanzada.

El diseño de la *app* que ha pensado uno de los informáticos del evento es el siguiente:



(2p) Implementa la estructura de la aplicación dada, usando POO

(8p) Condiciones para la construcción de las clases y otras cosas a tener en cuenta

- (0,5p) Debes crear una **excepción personalizada** que se lance en el método **validarCocheApto(Piloto piloto)** si un **Juez** intenta cambiar el estado de un coche a **APTO** sin que dicho coche esté previamente en estado **REVISADO**.
- (7,5p) Como todavía no podemos conectarnos a la base de datos, de momento, para hacer pruebas crearemos un programa principal **DriftSpainSeriesApp** con esta funcionalidad:

```
public class DriftSpainSeriesApp {

    public static ArrayList<Inscripcion> pilotosInscritos = new ArrayList<>();

    public static void main(String[] args) {

        System.out.println("=== BIENVENIDO A LA APP DEL DRIFT SPAIN SERIES DE VALENCIA ===");

        // a) crear 3 pilotos para las marcas SKODA, MERCEDES y TOYOTA

        // b) todos los pilotos se inscriben en la categoría PRO inscribirse()
        y guardamos sus inscripciones en la lista de pilotosInscritos
    }
}
```

```

        // c) crea al juez de la carrera

        // d) sólo 2 pilotos pagan la inscripción pagarInscripcion(Inscripcion
        inscripcion)

        // e) el juez valida validarInscripciones(Inscripcion inscripcion)
        que todas las inscripciones sean válidas (el piloto ha pagado). Aquellas
        inscripciones que no sean válidas, deben modificar el equipo del piloto a
        NO_VALIDO

        // f) crea los mecánicos asociados a cada piloto inscrito
        correctamente (su marca no es NO_VALIDO) y mételos en una nueva lista

        // g) los mecánicos revisan los coches de sus pilotos para dejarlos a
        punto para la carrera revisarVehiculo()

        // h) el juez comprueba el estado VALIDADO de los coches antes de
        iniciar la carrera y les da el APTO o NO_APTO: validarCocheApto(Piloto piloto)

        // i) el juez da luz verde a la carrera darSalida()

        // j) el juez puntúa puntuar(Piloto piloto) a todos los integrantes de
        la carrera (se recorre la lista de inscritos y por cada piloto con coche APTO
        calcula una puntuacion aleatoria

        // k) imprimimos ganador verGanador(CategoriaDrift categoria)
        según la puntuación otorgada por el juez

    }
}

```

- a) (0,5p) Crearemos 3 pilotos para las marcas *SKODA*, *MERCEDES* y *TOYOTA*. El constructor debe preguntar el **nombre** y el **país** de todos ellos por pantalla, asignarles una **puntuación** inicial de **0** y un **estado** del coche **PENDIENTE** por defecto.

```

// crear 3 pilotos para las marcas SKODA, MERCEDES y TOYOTA
Piloto skoda = new Piloto( marca: "SKODA");

```

```

"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:C:\Progr
=== BIENVENIDO A LA APP DEL DRIFT SPAIN SERIES DE VALENCIA ===
Inscribiendo participante...
Nombre:
IVAN
País:
ESPAÑA
Piloto IVAN de la marca SKODA creado.

Process finished with exit code 0
|

```

- b) (1p) Mediante el método **inscribirse()**, todos los *Pilotos* creados desde la app se apuntan a la carrera en *Categoría PRO* generando una nueva *Inscripción*.

```
// todos los pilotos se inscriben en la categoría PRO
Inscripcion skoda_insc = skoda.inscribirse();
```

El constructor de **Inscripcion** debe guardarse el **Piloto** que realiza la inscripción, preguntar la **categoría** de competición a participar y **pagado = false** por defecto.

```
=== BIENVENIDO A LA APP DEL DRIFT SPAIN SERIES DE VALENCIA ===
Inscribiendo participante...
Nombre:
IVAN
País:
ESPAÑA
Piloto IVAN de la marca SKODA creado.

Piloto IVAN realizando inscripción...
Elige una categoria [STREET, PRO, HYPER_DRIFT]:
PRO
Piloto IVAN inscrito en PRO. Pendiente de pago: 150.0€

Process finished with exit code 0
```

En caso de elegir una categoría de competición que no exista, el programa debe mostrar "**Categoría incorrecta. Elige una existente.**" y volver a preguntar al usuario hasta que introduzca una bien:

```
Piloto IVAN realizando inscripción...
Elige una categoria [STREET, PRO, HYPER_DRIFT]:
HOLA
Categoría incorrecta. Elige una existente.
Elige una categoria [STREET, PRO, HYPER_DRIFT]:
|
```

Ve guardando las inscripciones en la lista de **pilotosInscritos** de la app.

c) (0,25p) Crea al **Juez** de la carrera en la app.

```
Inscribiendo participante...
Nombre:
MALO
País:
FRANCIA
Juez MALO creado.
```

- d) (0,5p) Haz que solamente 2 de los 3 pilotos paguen la inscripción con el método ***pagarInscripcion(Inscripcion inscripcion)***. Este método se encargará de modificar el atributo **pagado** de la inscripción del *Piloto* a **true**.

```
Piloto IVAN pagando 150.0 € por la inscripción en categoría PRO
Piloto PATRI pagando 150.0 € por la inscripción en categoría PRO
```

TRUCO: Cuando termines este paso, imprime la lista de ***pilotosInscritos*** para comprobar el estado del atributo **pagado** para cada uno.

```
[Inscripcion(piloto=Participante(nombre=IVAN, pais=ESPAÑA), categoria=PRO, pagado=true), Inscripcion(piloto=Participante(nombre=PATRI, pais=ESPAÑA), cate
```

```
[Inscripcion(piloto=Participante(nombre=IVAN, pais=ESPAÑA), categoria=PRO, pagado=true),
Inscripcion(piloto=Participante(nombre=PATRI, pais=ESPAÑA), categoria=PRO, pagado=true),
Inscripcion(piloto=Participante(nombre=MICHAEL, pais=EEUU), categoria=PRO,
pagado=false)]
```

- e) (1p) El *Juez* comprueba desde la app que todas las inscripciones sean válidas (el piloto ha pagado) con el método ***validarInscripciones(Inscripcion inscripcion)***. Para aquellas inscripciones que no sean válidas, deberemos modificar el valor de la **marca** del piloto a **NO_VALIDA**

```
Validando inscripciones...
Piloto IVAN: Inscripción válida.
Piloto PATRI: Inscripción válida.
La inscripción no está completada por el piloto MICHAEL
```

- f) (1p) Crea desde la app a los *Mecánicos* sólo para los *Pilotos* correctamente inscritos (que no tengan como **marca NO_VALIDO**) y mételos en una nueva lista.

```
Creando mecánico para IVAN:

Inscribiendo participante...
Nombre:
JOSE LUIS
País:
ESPAÑA
Mecánico JOSE LUIS creado y asignado al piloto IVAN

Creando mecánico para PATRI:

Inscribiendo participante...
Nombre:
ISMAEL
País:
MARRUECOS
Mecánico ISMAEL creado y asignado al piloto PATRI
```

- g) (0,5p) Los mecánicos revisan mediante el método **revisarVehiculo()** los coches de sus pilotos para dejarlos a punto para la carrera. Este método solamente se encarga de modificar el estado del coche del piloto a **REVISADO**.

```
Mecánico JOSE LUIS revisando vehículo de IVAN  
  
Mecánico ISMAEL revisando vehículo de PATRI
```

- h) (1p) El juez comprueba el estado **VALIDADO** de los coches de todos los pilotos correctamente inscritos antes de iniciar la carrera mediante el método **validarCocheApto(Piloto piloto)**. Si el coche se encuentra en estado **REVISADO** les modifica el **estado** a **APTO**, y si no, a **NO_APTO**.

```
Juez validando coches...  
Coche del piloto IVAN APTO.  
Coche del piloto PATRI APTO.
```

- i) (0,25p) El juez da la salida a la carrera mediante el método **darSalida()**:

```
Juez MALO dando salida a la carrera en 3... 2... 1
```

- j) (1p) El juez puntúa con el método **puntuar(Piloto piloto)** sólo a aquellos pilotos que hayan conseguido el estado **APTO** en sus coches y están en carrera. Debe generar un número aleatorio del 1 al 10 y modificarle la **puntuación** a cada *Piloto*.

```
Juez puntuando a IVAN con 5 puntos.  
  
Juez puntuando a PATRI con 2 puntos.
```

- k) (0,5p) Para finalizar, la app imprime al piloto ganador de la carrera y su categoría.

```
El piloto ganador en categoría PRO ha sido PATRI
```

Salida de ejemplo

```
"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:C:\Program Files
=== BIENVENIDO A LA APP DEL DRIFT SPAIN SERIES DE VALENCIA ===

Inscribiendo participante...
Nombre:
IVAN
País:
ESPAÑA
Piloto IVAN de la marca SKODA creado.

Inscribiendo participante...
Nombre:
PATRI
País:
ESPAÑA
Piloto PATRI de la marca MERCEDES creado.

Inscribiendo participante...
Nombre:
MICHAEL
País:
EEUU
Piloto MICHAEL de la marca TOYOTA creado.

Piloto IVAN realizando inscripción...
Elige una categoria [STREET, PRO, HYPER_DRIFT]:
PRO
Piloto IVAN inscrito en PRO. Pendiente de pago: 150.0€

Piloto PATRI realizando inscripción...
Elige una categoria [STREET, PRO, HYPER_DRIFT]:
PRO
Piloto PATRI inscrito en PRO. Pendiente de pago: 150.0€

Piloto MICHAEL realizando inscripción...
Elige una categoria [STREET, PRO, HYPER_DRIFT]:
PRO
Piloto MICHAEL inscrito en PRO. Pendiente de pago: 150.0€
```

Inscribiendo participante...

Nombre:

MALO

País:

FRANCIA

Juez MALO creado.

Piloto IVAN pagando 150.0 € por la inscripción en categoría PRO

Piloto PATRI pagando 150.0 € por la inscripción en categoría PRO

[Inscripcion(piloto=Participante(nombre=IVAN, pais=ESPAÑA), categoria=PRO,

Juez validando inscripciones...

Piloto IVAN: Inscripción válida.

Piloto PATRI: Inscripción válida.

La inscripción no está completada por el piloto MICHAEL

Creando mecánico para IVAN:

Inscribiendo participante...

Nombre:

JOSE LUIS

País:

ESPAÑA

Mecánico JOSE LUIS creado y asignado al piloto IVAN

Creando mecánico para PATRI:

Inscribiendo participante...

Nombre:

ISMAEL

País:

MARRUECOS

Mecánico ISMAEL creado y asignado al piloto PATRI

Mecánico JOSE LUIS revisando vehículo de IVAN

Mecánico ISMAEL revisando vehículo de PATRI


```
Juez validando coches...
Coche del piloto IVAN APT0.
Coche del piloto PATRI APT0.

Juez MAL0 dando salida a la carrera en 3... 2... 1

Juez puntuando a IVAN con 7 puntos.

Juez puntuando a PATRI con 3 puntos.

El piloto ganador en categoría PRO ha sido IVAN

Process finished with exit code 0
```